

Ingeniería del Software de Componentes y Sistemas Distribuidos

SEGUNDA PRUEBA PRÁCTICA – PRAC 2

Presentación

En la PRAC 1 se ha diseñado una solución para el caso **CultureInDaHouse** basada en microservicios, usando modelos de representación y patrones de diseño. En esta PRAC 2 se pide implementar un subconjunto de este diseño a partir de programar 2 microservicios considerando diferentes aspectos, como, por ejemplo: estado propio, comunicación síncrona, lógica de negocio, consistencia de los datos y despliegue en local.

Para elaborar esta práctica se formulan 2 ejercicios a partir del caso **CultureInDaHouse**: 1) implementar dos de los microservicios con diseñados en la PRAC1, según un conjunto de consideraciones técnicas y la documentación de las evidencias; 2) razonar cómo resolver el acceso a los microservicios desde diferentes clientes.

La PRAC 2 abarca los contenidos del libro "Microservices Patterns" (ver en el apartado Recursos los capítulos que hay que estudiar).

Competencias

En esta PRAC 2 se trabaja las siguientes competencias del Grado en Ingeniería Informática:

- Saber construir aplicaciones informáticas mediante técnicas de desarrollo, integración y reutilización.
- Saber proponer y evaluar diferentes alternativas tecnológicas para resolver un problema concreto.

Objetivos

Los objetivos de la PRAC 2 son:

- Implementar microservicios centrados en la lógica de negocio y el acceso a datos.
- Desplegar microservicios con estado propio e intercomunicación síncrona.
- Aplicar conceptos básicos de la arquitectura de microservicios.
- Dominar las tecnologías básicas necesarias por la implementación de microservicios.

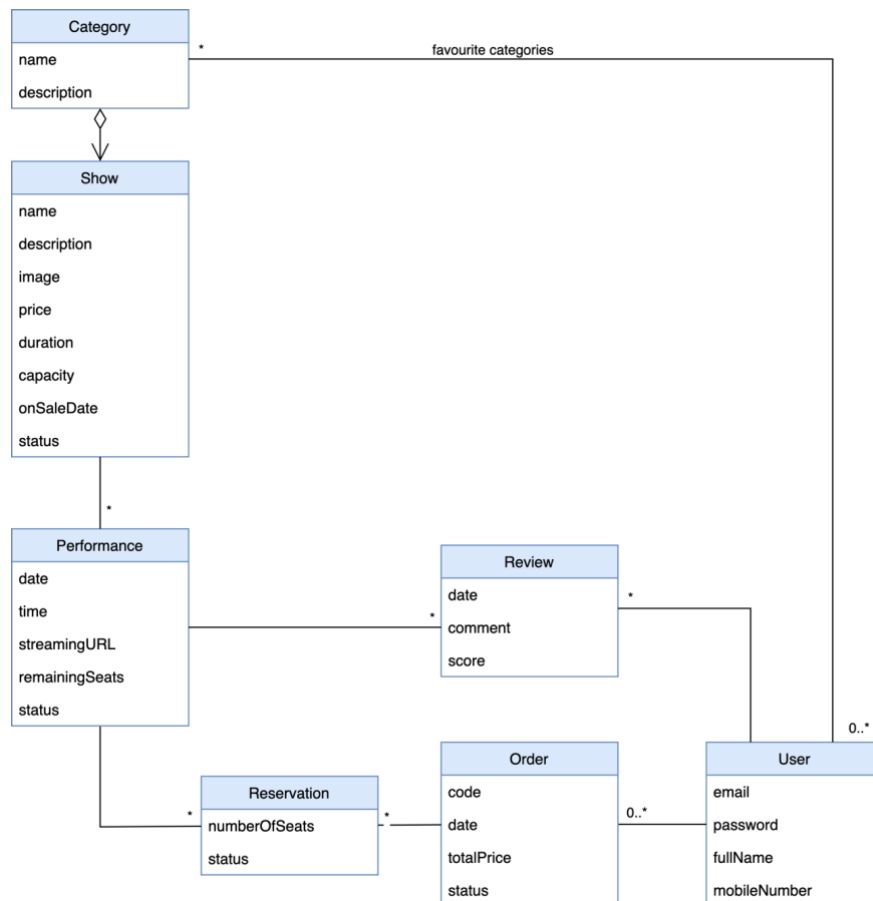
Descripción de la PRAC 2

Ejercicio 1 (8 puntos)

El Ejercicio 1 consta de dos partes, la primera parte consistirá en la implementación de un software, y la segunda parte en la documentación de las evidencias de lo que se ha implementado. A continuación, se describen las tareas concretas a realizar en cada una de las dos partes.

Primera parte (6 puntos)

Consiste en implementar un subconjunto de los microservicios que se han diseñado en la PRAC1. Debe basarse en la solución publicada por la PRAC anterior y por tanto se parte del siguiente diagrama del modelo general del dominio, del que se pueden extraer las principales entidades (con sus atributos) y relaciones:



El subconjunto de servicios a implementar se compone de los servicios *Show Catalog* y *Notification*:

- El microservicio *Show Catalog* servirá tanto para dar cobertura a las diversas operaciones que los administradores de la plataforma necesitan para gestionar los contenidos (crear/eliminar actas, categorías y actuaciones, etc.) como para las consultas del catálogo de actos que realizarán los usuarios .
- El microservicio *Notification* tiene la finalidad de aumentar el “engagement” de los usuarios de la plataforma y debe funcionar de forma que cuando se cree un nuevo acto cultural, se envíe una notificación (correo electrónico) a todos los usuarios que tengan cómo favorita cualquiera de las categorías del acto. Dado que **CultureInDaHouse** todavía no ha podido obtener el consentimiento de los usuarios para recibir correos electrónicos, tanto la interacción con el microservicio de usuarios como el envío de los correos en sí deberán simularse por medio de un *mock*.

De acuerdo con lo descrito, estos dos microservicios tendrán como mínimo las siguientes operaciones:

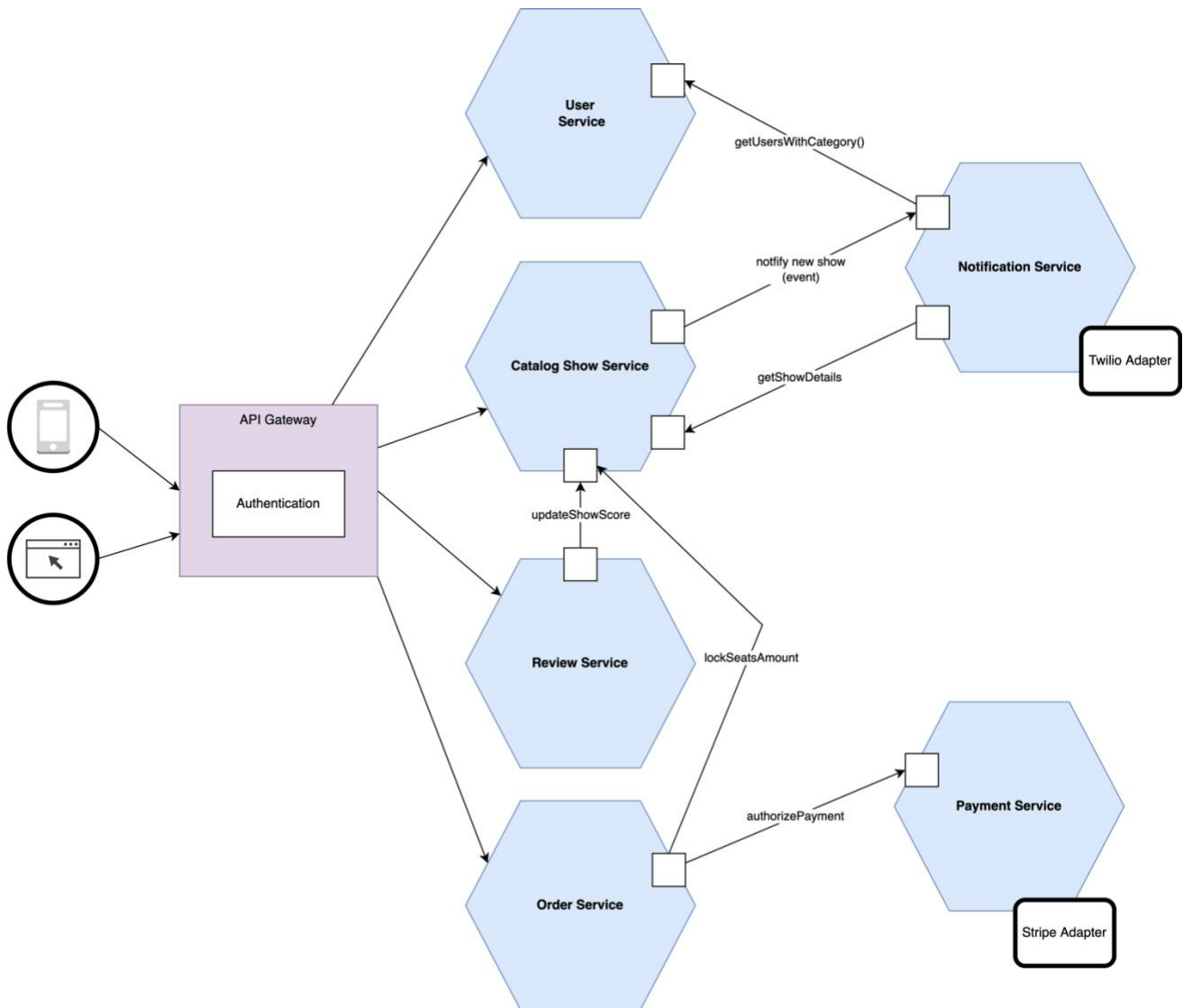
Microservicio *Show Catalog* (catálogo de actos)

- crear categoría
- crear acto
- crear actuación
- eliminar categoría
- eliminar acto
- eliminar actuación
- consulta de categorías
- consulta de shows por nombre
- consulta de shows por categoría
- consulta del detalle de un show
- consulta de actuaciones de un show

Microservicio *Notification* (notificaciones a usuarios)

- enviar notificación

En el siguiente diagrama se puede ver cómo debe ser la colaboración entre los diversos servicios del sistema:



Consideraciones previas

Para poder conseguir la flexibilidad, resiliencia y rendimiento que caracteriza esta arquitectura, es necesario cumplir una serie de requerimientos:

- Cada microservicio:
 - tiene que poder ejecutarse independientemente,
 - tiene que tener estado propio (y acceder de manera independiente a la BBDD),
 - implementar solo su lógica de negocio,
 - exponer su contrato (definición de la API),
 - gestionar su propio *log*.

- Tiene que existir una comunicación entre los diversos microservicios, pero siempre por medio de mensajes.
- No será necesario ningún tipo de autenticación, se da por hecho que esta parte de la funcionalidad del sistema la proporciona la API Gateway (que no se tiene que implementar).
- No se tiene que implementar ningún tipo de funcionalidad de *Service Discovery & Registration*.
- Es posible que en algún momento se necesite la colaboración de otro servicio diferente de los dos que se han implementado (por. ej. *User Service*). En este caso hay que utilizar un *mock* para simular las llamadas y los valores devueltos por este servicio.

Plataforma de desarrollo

- Entorno de desarrollo: IntelliJ IDEA / Eclipse
- JDK: OpenJDK 11
- Framework: Spring / Spring Boot
- Gestión de dependencias y build: Maven
- Servidor HTTP: Tomcat (integrado por defecto con Spring Boot)
- Librería llamadas REST: Jersey
- Comunicación entre servicios: Apache Kafka
- BBDD: PostgreSQL
- Implementación ORM: Hibernate
- Generación de la documentación del API OpenAPI v3: springdoc-openapi
- Log: Log4j2
- Mocks: Mockneat

Punto de partida y recursos

Para tener un punto de partida común y homogéneo, y poder centrar los esfuerzos en el desarrollo de los microservicios, junto con este enunciado se proporciona un enlace a un repositorio público (<https://github.com/ppinedar/epcsd-spring>) con los siguientes recursos:

- Un archivo *docker-compose.yml* para instalar la infraestructura básica para la implementación de este ejercicio y un archivo *README.md* con instrucciones para arrancarla.
- Dos esqueletos de aplicación Spring Boot con las funcionalidades básicas para generar un archivo *java jar* ejecutable con todas las dependencias contenidas al proyecto.
- Una serie de enlaces a documentación, guías y tutoriales relativos a la implementación de los microservicios tratados (que se colgarán en el aula de Laboratorio).

Cada uno de los esqueletos de aplicación que se proporcionan son un proyecto Spring Boot completo pre-configurado con las dependencias mínimas necesarias para hacer toda la implementación que se pide en este enunciado. Esto no significa que no se puedan añadir otras dependencias si os parece adecuada, pero con el que hay basta para hacer toda la implementación.

Con los contenedores del archivo *docker-compose* arrancados, ambos esqueletos de microservicio tendrían que poder ponerse en marcha y conectar a la infraestructura tal como se entregan con esta

PRAC2. Además, aprovechando que los esqueletos llevan su propio servidor HTTP empotrado, se ha incluido un módulo *springdoc-openapi* que auto-genera y publica el archivo de definición del microservicio en formato *JSON/OpenAPI v3*, así como una pequeña interfaz web *SwaggerUI* que permite hacer llamadas a las operaciones de los microservicios.

Las direcciones para acceder a estas funcionalidades son:

- archivos de definición de los servicios: <http://localhost:18081/v3/api-docs/> y <http://localhost:18082/v3/api-docs/>
- interfaces web *SwaggerUI* por pruebas: <http://localhost:18081/swagger-ui/index.html> y <http://localhost:18082/swagger-ui/index.html>

Cómo se puede ver, estos ejemplos forman una base bastante completa desde donde empezar el desarrollo y, por lo tanto, se tendría que evitar hacer modificaciones en sus componentes básicos (excepto pequeñas configuraciones, etc.), y solo añadir las funcionalidades que se piden en este enunciado. Encontraréis más información y apoyo técnico en el Laboratorio de la asignatura.

Entregable

Especificaciones técnicas de la aplicación a entregar

El entregable de esta parte del ejercicio será un único archivo ZIP que tiene que contener:

- Un archivo *JAR java* ejecutable (generado por el propio esqueleto del proyecto entregado) para cada uno de los servicios, que contendrá todas las clases y librerías necesarias (dependencias) para levantar el servicio en cuestión. Este archivo se puede generar a partir del esqueleto de aplicación proporcionado con una ejecución *maven* y los “goals” *clean package* (o simplemente ejecutando *mvn clean package* por línea de comandos).
- Un archivo *readme.txt* con la siguiente información:
 - instrucciones para poner en marcha la infraestructura necesaria para la plataforma (en caso de que sea diferente a la que se proporciona con *docker compose*)
 - cualquier configuración necesaria para poder ejecutar los servicios
 - el comando concreto para ejecutar cada servicio por separado (si no se ha cambiado nada, tendrían que ser *java -jar showcatalog-VERSION.jar* y *java -jar notification-VERSION.jar*)

Solución

Se puede acceder a una implementación de referencia de los microservicios especificados desde los repositorios públicos GitHub localizados en las direcciones:

<https://github.com/ppinedar/epcsd-spring-showcatalog-solucio>
<https://github.com/ppinedar/epcsd-spring-notification-solucio>

Segunda parte (2 puntos)

Para poder demostrar que se han logrado los objetivos de la implementación, tendréis que proporcionar evidencias del correcto funcionamiento de los microservicios. Para elaborar las evidencias se puede usar cualquier de los siguientes métodos por orden de preferencia (podéis consultar el Laboratorio por más información y apoyo):

- Utilizar la aplicación *Postman* para importar el archivo de definición del servicio (accesible desde la ruta *v3/api-docs* de los dos microservicios) y generar la colección de forma que se pueda hacer al menos un ejemplo de cada tipo de llamada una vez puestos en marcha los servicios.
- Un vídeo (*screencast*) donde se pueda ver el funcionamiento de todo el sistema: arranque, ejecución de los microservicios, ejecución y resultado de las llamadas a los servicios en tiempo real usando la interfaz *SwaggerUI* integrada en los esqueletos de aplicación proporcionados (accesibles en las direcciones: <http://localhost:18081/swagger-ui/index.html> y <http://localhost:18082/swagger-ui/index.html>).
- Una lista de pedidos CURL con todos los parámetros necesarios para ejecutar las llamadas (tienen que estar preparadas de forma que, con la infraestructura y los microservicios arrancados, solo haya que hacer *copy&paste* en una consola para hacer la comprobación).

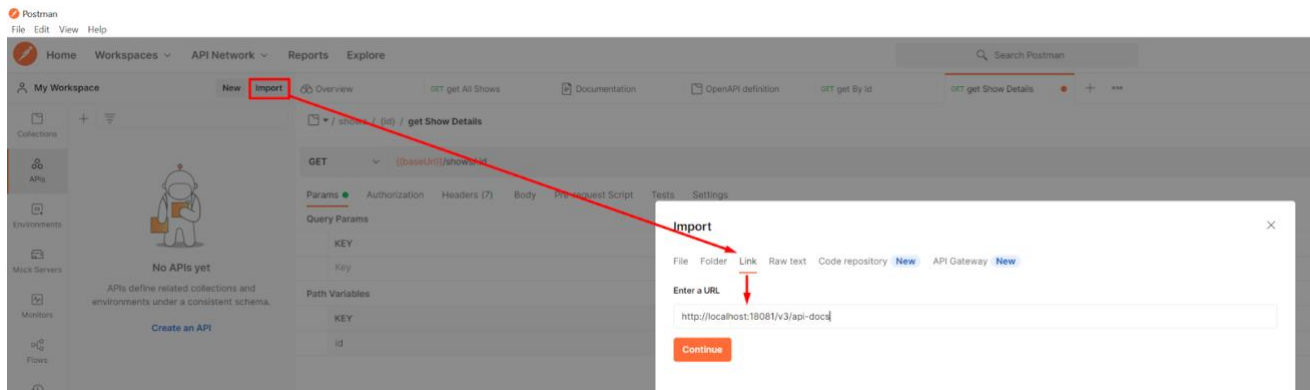
Sea cual sea la opción escogida, las evidencias tienen que cubrir como mínimo un ejemplo de cada una de las operaciones implementadas (petición y respuesta).

Solución

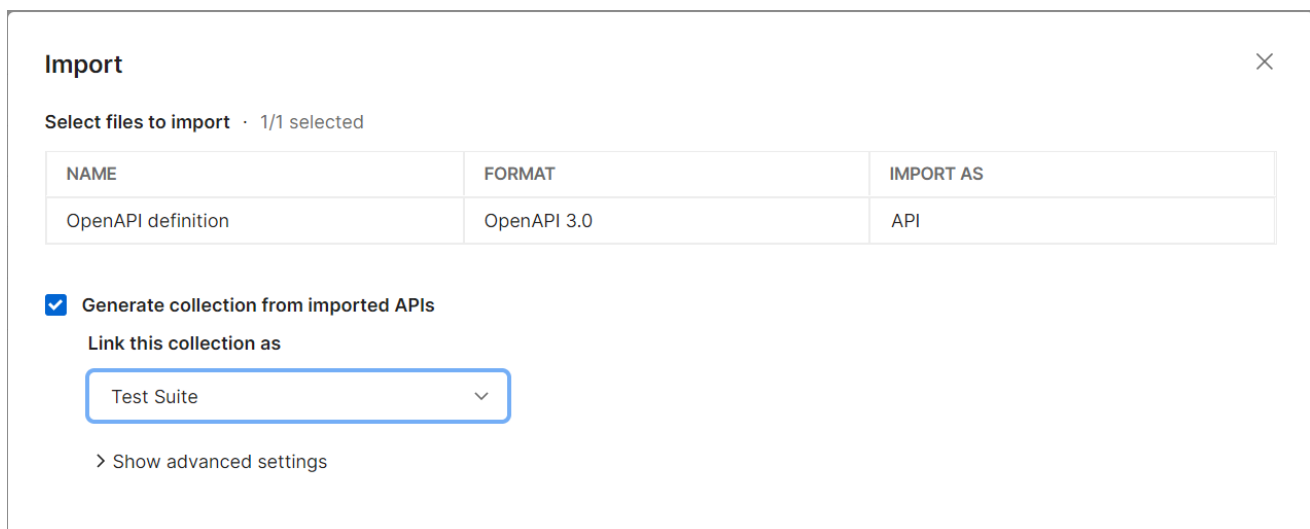
Una vez hecha la implementación, se puede obtener el “contrato” o la definición de cada uno de los servicios a la dirección *v3/api-docs*.

```
{
  "openapi": "3.0.1",
  "info": {
    "title": "OpenAPI definition",
    "version": "v1",
    "servers": [
      {
        "url": "http://localhost:18081",
        "description": "Generated server url"
      }
    ],
    "paths": {
      "/show/{id}": {
        "get": {
          "tags": [
            "show-controller"
          ],
          "operationId": "getById",
          "parameters": [
            {
              "name": "id",
              "in": "path",
              "required": true,
              "schema": {
                "type": "integer",
                "format": "int64"
              }
            }
          ],
          "responses": {
            "200": {
              "description": "OK",
              "content": {
                "application/json": {
                  "schema": {
                    "$ref": "#/components/schemas/show"
                  }
                }
              }
            }
          }
        }
      },
      "/delete": {
        "delete": {
          "tags": [
            "show-controller"
          ],
          "operationId": "delete",
          "parameters": [
            {
              "name": "id",
              "in": "path",
              "required": true,
              "schema": {
                "type": "integer",
                "format": "int64"
              }
            }
          ],
          "responses": {
            "200": {
              "description": "OK",
              "content": {
                "application/json": {
                  "schema": {
                    "$ref": "#/components/schemas/show"
                  }
                }
              }
            }
          }
        }
      },
      "/category/{id}": {
        "get": {
          "tags": [
            "category-controller"
          ],
          "operationId": "getById",
          "parameters": [
            {
              "name": "id",
              "in": "path",
              "required": true,
              "schema": {
                "type": "integer",
                "format": "int64"
              }
            }
          ],
          "responses": {
            "200": {
              "description": "OK",
              "content": {
                "application/json": {
                  "schema": {
                    "$ref": "#/components/schemas/category"
                  }
                }
              }
            }
          }
        }
      },
      "/update": {
        "put": {
          "tags": [
            "category-controller"
          ],
          "operationId": "update",
          "parameters": [
            {
              "name": "id",
              "in": "path",
              "required": true,
              "schema": {
                "type": "integer",
                "format": "int64"
              }
            }
          ],
          "requestBody": {
            "content": {
              "application/json": {
                "schema": {
                  "$ref": "#/components/schemas/category"
                }
              }
            }
          },
          "responses": {
            "200": {
              "description": "OK",
              "content": {
                "application/json": {
                  "schema": {
                    "$ref": "#/components/schemas/category"
                  }
                }
              }
            }
          }
        }
      },
      "/show": {
        "get": {
          "tags": [
            "show-controller"
          ],
          "operationId": "getById",
          "parameters": [
            {
              "name": "id",
              "in": "path",
              "required": true,
              "schema": {
                "type": "integer",
                "format": "int64"
              }
            }
          ],
          "responses": {
            "200": {
              "description": "OK",
              "content": {
                "application/json": {
                  "schema": {
                    "$ref": "#/components/schemas/show"
                  }
                }
              }
            }
          }
        }
      },
      "/create": {
        "post": {
          "tags": [
            "show-controller"
          ],
          "operationId": "create",
          "requestBody": {
            "content": {
              "application/json": {
                "schema": {
                  "$ref": "#/components/schemas/show"
                }
              }
            }
          },
          "responses": {
            "200": {
              "description": "OK",
              "content": {
                "application/json": {
                  "schema": {
                    "$ref": "#/components/schemas/show"
                  }
                }
              }
            }
          }
        }
      },
      "/category": {
        "get": {
          "tags": [
            "category-controller"
          ],
          "operationId": "getById",
          "parameters": [
            {
              "name": "id",
              "in": "path",
              "required": true,
              "schema": {
                "type": "integer",
                "format": "int64"
              }
            }
          ],
          "responses": {
            "200": {
              "description": "OK",
              "content": {
                "application/json": {
                  "schema": {
                    "$ref": "#/components/schemas/category"
                  }
                }
              }
            }
          }
        }
      },
      "/create": {
        "post": {
          "tags": [
            "category-controller"
          ],
          "operationId": "create",
          "requestBody": {
            "content": {
              "application/json": {
                "schema": {
                  "$ref": "#/components/schemas/category"
                }
              }
            }
          },
          "responses": {
            "200": {
              "description": "OK",
              "content": {
                "application/json": {
                  "schema": {
                    "$ref": "#/components/schemas/category"
                  }
                }
              }
            }
          }
        }
      }
    }
  }
}
```

Este archivo *JSON* se puede importar a una herramienta externa como *Postman*, desde dónde será mucho más sencillo realizar todas las pruebas necesarias para verificar el funcionamiento correcto de los diferentes servicios.



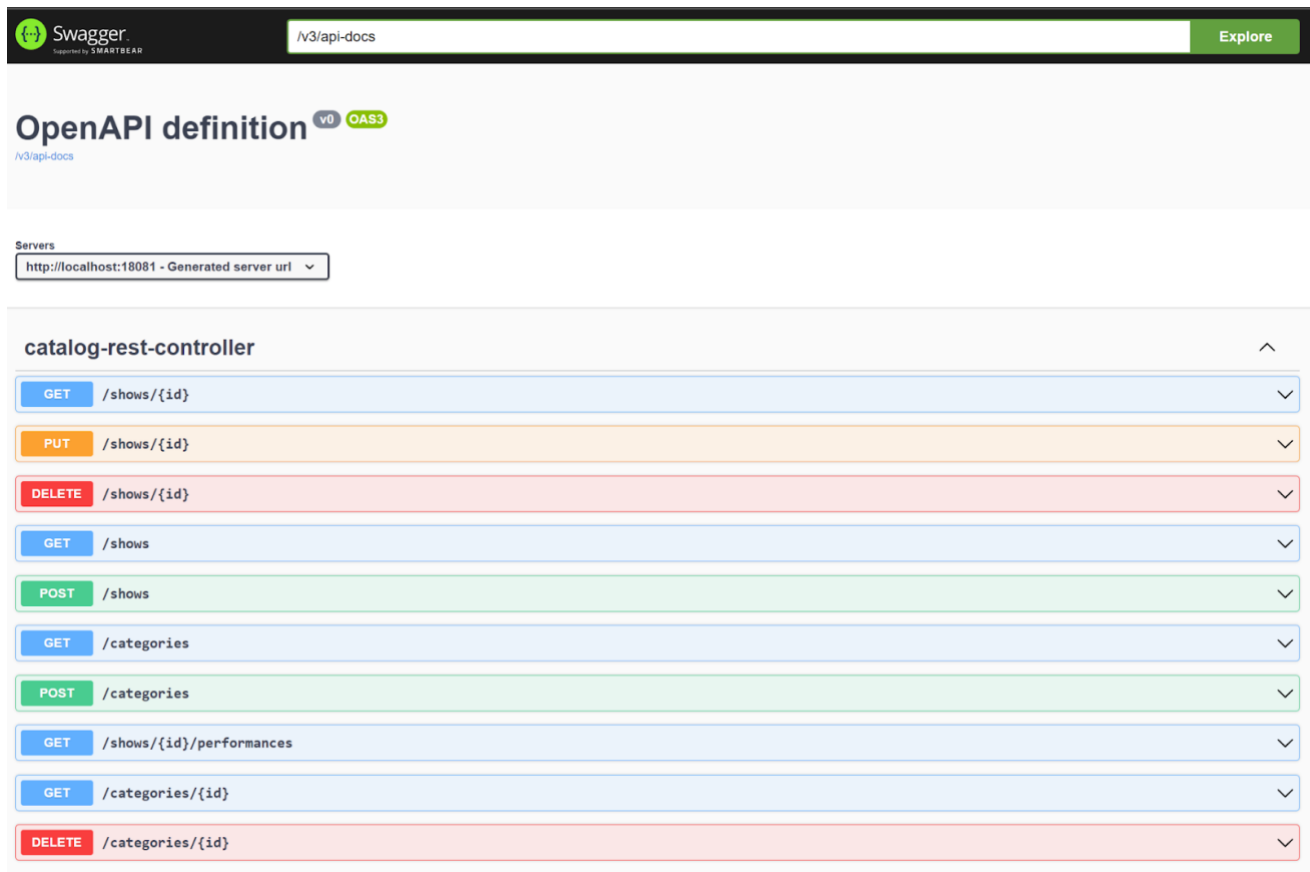
Desde esta aplicación se pueden generar llamadas contra los servicios implementados en la primera parte para ver las peticiones/respuestas.



Postman interface showing a GET request to `/shows`. The response is a JSON array of show objects. A red box highlights the `GET find Shows` endpoint in the left sidebar, and a red arrow points to the `GET find Shows` tab in the top bar.

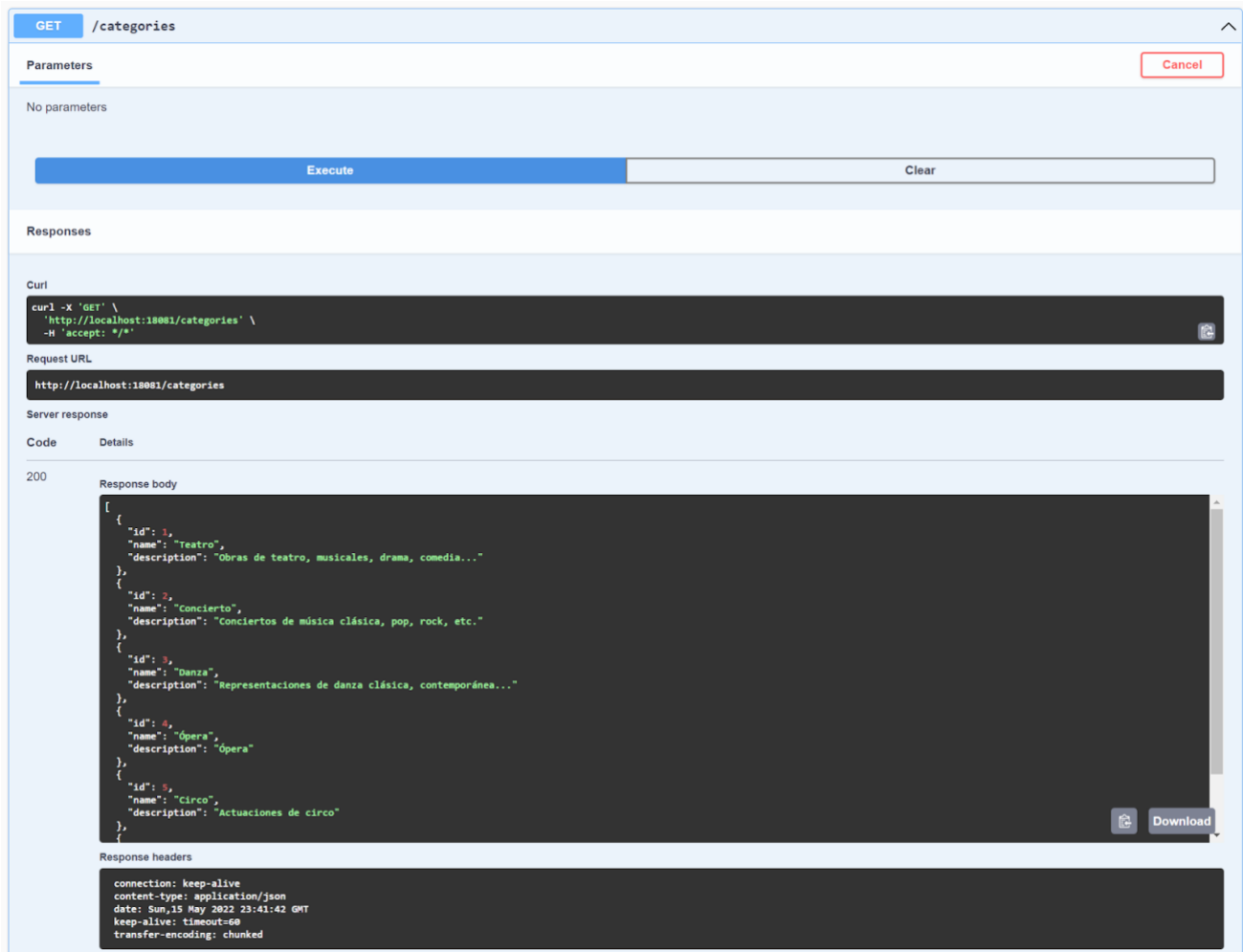
Postman interface showing a POST request to `{{baseUri}}/shows`. The request body is a JSON object. The response is a JSON object with a `show` field containing a new show ID. A red box highlights the `POST create Show` endpoint in the left sidebar, and a red arrow points to the `POST create Show` tab in the top bar. Another red box highlights the `show` field in the response body, and a red arrow points to the `show` field in the response body.

Alternativamente, también se puede usar la propia interfaz SwaggerUI integrada en los esqueletos de aplicación:



The image shows the SwaggerUI interface for an OpenAPI definition. The top bar includes the Swagger logo, a search bar with the text "/v3/api-docs", and an "Explore" button. Below the bar, the text "OpenAPI definition" is displayed with version tags "v0" and "OAS3". A "Servers" section shows a dropdown menu with the selected server "http://localhost:18081 - Generated server url". The main content area is titled "catalog-rest-controller" and lists ten API endpoints, each with a method (GET, PUT, DELETE, POST) and a path. The endpoints are color-coded: GET (blue), PUT (orange), DELETE (red), and POST (green). Each endpoint has a dropdown arrow on the right.

Method	Path
GET	/shows/{id}
PUT	/shows/{id}
DELETE	/shows/{id}
GET	/shows
POST	/shows
GET	/categories
POST	/categories
GET	/shows/{id}/performances
GET	/categories/{id}
DELETE	/categories/{id}



The screenshot shows a REST client interface with the following details:

- Method:** GET
- Path:** /categories
- Parameters:** No parameters
- Request URL:** http://localhost:18081/categories
- Response Code:** 200
- Response Body (JSON):**

```
[
  {
    "id": 1,
    "name": "Teatro",
    "description": "Obras de teatro, musicales, drama, comedia..."
  },
  {
    "id": 2,
    "name": "Concierto",
    "description": "Conciertos de música clásica, pop, rock, etc."
  },
  {
    "id": 3,
    "name": "Danza",
    "description": "Representaciones de danza clásica, contemporánea..."
  },
  {
    "id": 4,
    "name": "Ópera",
    "description": "Ópera"
  },
  {
    "id": 5,
    "name": "Circo",
    "description": "Actuaciones de circo"
  }
]
```
- Response Headers:**

```
connection: keep-alive
content-type: application/json
date: Sun, 15 May 2022 23:41:42 GMT
keep-alive: timeout=60
transfer-encoding: chunked
```

O incluso simplemente comandos CURL (a pesar de que entonces los entregables serán de tipo texto).

```
curl -X 'GET' \
  'http://localhost:18081/show/' \
  -H 'accept: */*'
```

Ejercicio 2 (2 puntos)

Un ayuntamiento cercano ha manifestado su interés en publicar en su boletín semanal las actuaciones que tendrán lugar en su pequeño municipio. Paralelamente, una empresa de venta online de entradas nos ofrece poder publicar eventos en su buscador de espectáculos a cambio de un % como comisión sobre el precio final de venta.

Con el objetivo de mejorar los resultados económicos aumentando la asistencia a los diferentes espectáculos disponibles en **CultureInDaHouse**, el CEO ha pactado acuerdos para permitir el acceso a la información contenida en nuestros servicios desde estas dos plataformas externas.

Como se ve, el repentino éxito de la plataforma **CultureInDaHouse** ha provocado la aparición de nuevos casos de uso no contemplados previamente, pero afortunadamente, la arquitectura de microservicios le proporciona flexibilidad y herramientas para poder afrontar estos escenarios. Razona, en base al escenario planteado en este ejercicio, qué patrón podría aplicarse para resolver el acceso a los microservicios de nuestra plataforma desde diferentes clientes o sistemas externos de manera que se pueda acceder a la información acordada (catálogo de actos y actuaciones).

Solución

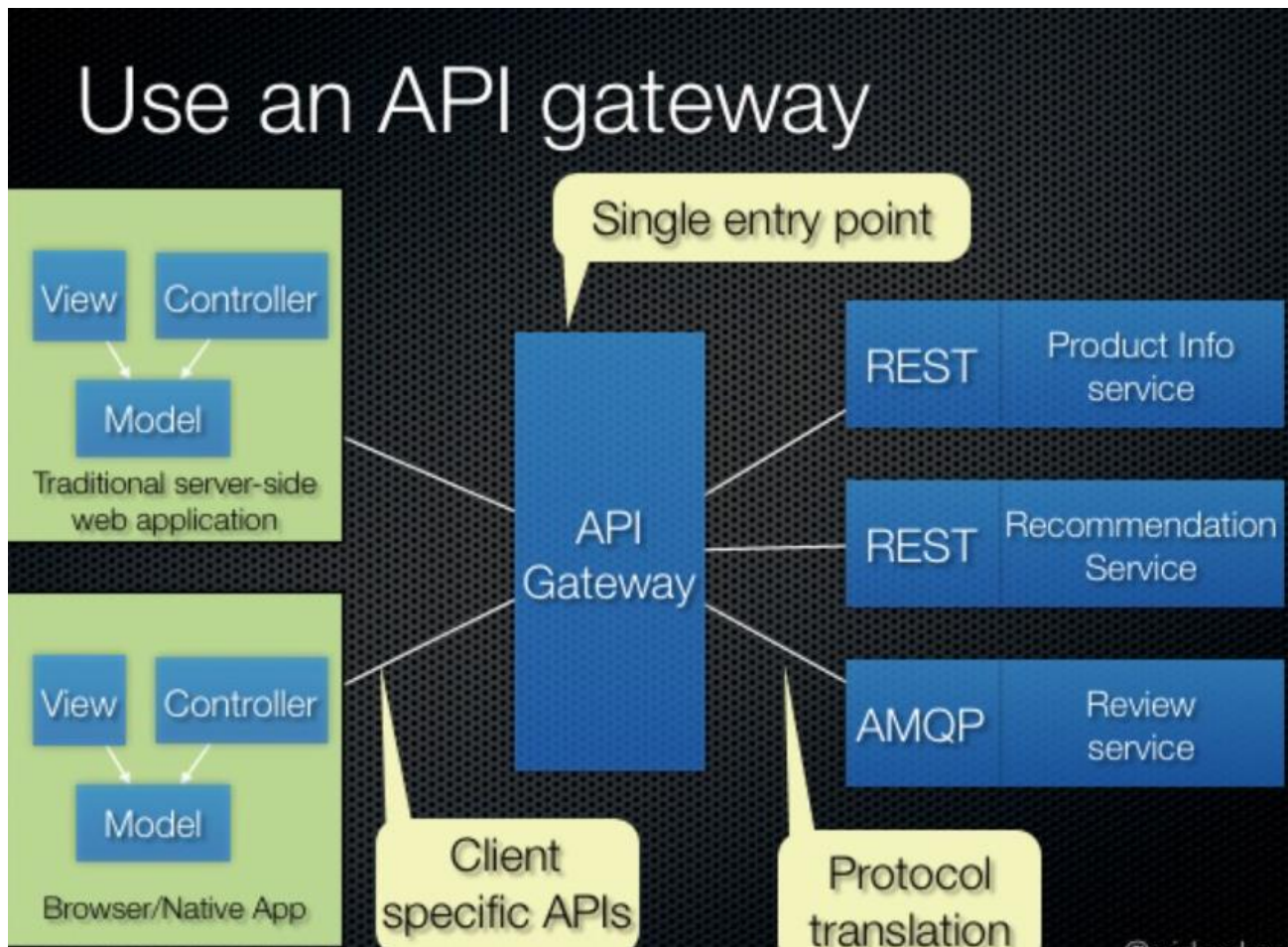
No es prudente exponer nuestros servicios directamente en el exterior, por diversas razones:

- Seguridad.
- Estaríamos exponiendo más datos y operaciones de los realmente necesarios (no es necesario que los sistemas externos hagan consultas referentes a los usuarios, y de hecho, podría constituir un delito, por ejemplo)
- Cada vez que actualicemos el servicio, los usuarios externos también tendrán que actualizar sus clientes.
- Etc.

Aparte de esto, si en el futuro se quiere dar acceso a otros sistemas externos, necesitaremos un medio para controlar el acceso.

El patrón que nos sirve para resolver esta nueva problemática es API Gateway. Una API Gateway es una pasarela que sirve de punto único de acceso a una aplicación desde el exterior.

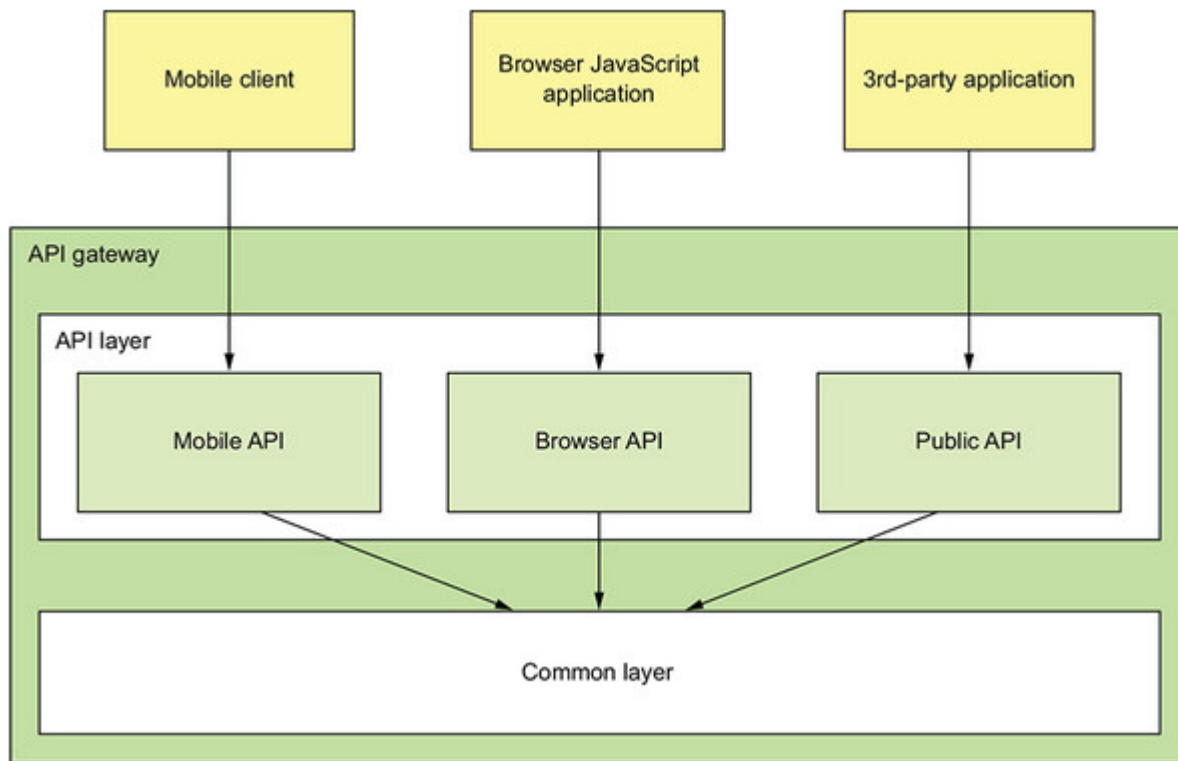
La aplicación de este patrón nos permite ofrecer una fachada al exterior que encapsula toda la arquitectura interna de forma similar a cómo se hace aplicando el patrón *Fachada*, bastante conocido dentro de la programación orientada a objetos.



Las funciones principales de este patrón son:

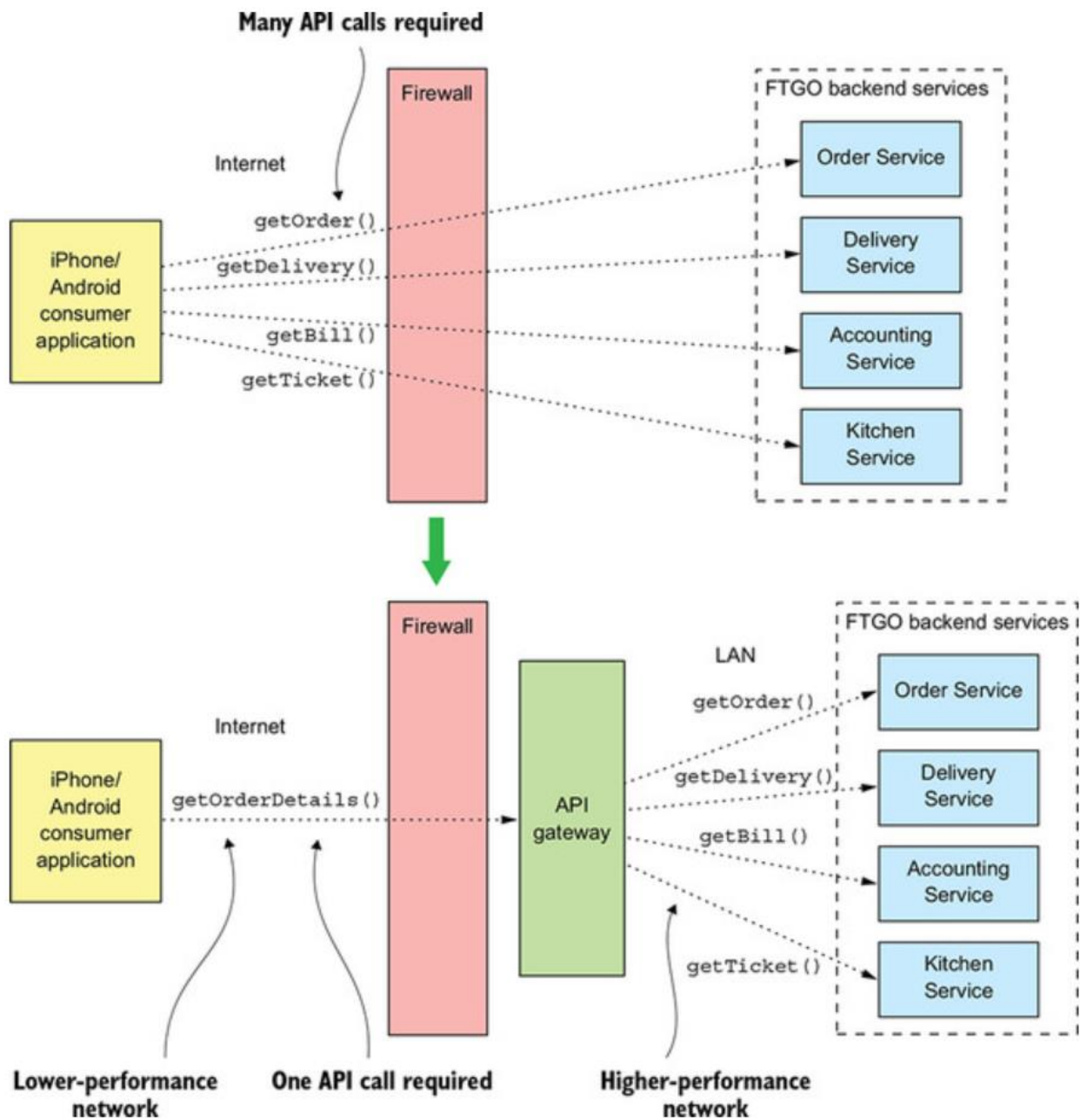
- El enrutamiento de las peticiones desde el punto de acceso único hacia los varios servicios internos de que disponemos.
- La composición de las varias APIs que hay en nuestro sistema.

No obstante, también hay otras ventajas. La aplicación de este patrón nos aporta muchísima flexibilidad y granularidad, puesto que podemos optar para ofrecer una API concreta según la plataforma a la que queremos dar servicio, o diferentes APIs por diferentes clientes concretos. Finalmente, incluso se podría ofrecer una API de acceso público por aplicaciones de terceros, como hacen, por ejemplo, varias aplicaciones de redes sociales (Twitter, Instagram, etc.).



También es posible, gracias a esta pasarela, aplicar transformaciones a las peticiones, aplicar *cachés*, recoger métricas diversas, etc.

En el siguiente diagrama se puede apreciar el uso de este patrón aplicado en el ejemplo FTGO del libro *Microservices Patterns*:



Referencias:

- Microservices Patterns (Richardson). Chapter 8: "External API patterns".
- API Gateway: <https://microservices.io/patterns/apigateway.html>

Recursos

Recursos Básicos

- Libro "Microservices Patterns" (Chris Richardson): Chapter 6: "Developing business logic with event sourcing" (excepto sección 6.3).
- Libro "Microservices patterns" (Richardson): Chapter 7: "Implementing queries in a microservice architecture"
- Libro "Microservices patterns" (Richardson): Chapter 8: "External APIO patterns" (excepto sección 8.3).

Recursos Complementarios

- Repositorio de código fuente del ejemplo del libro Microservices Patterns. <https://github.com/microservices-patterns/ftgo-application>.
- Ejemplo de programación de microservicios. <https://github.com/vmudigal/microservices-sample>

Criterios de evaluación

- La PRAC 2 se tiene que resolver de forma **estrictamente individual**. En caso de detectar copias o cualquier otra forma de fraude académico, se penalizará la prueba con una D como nota.
- El peso de cada ejercicio está indicado en el enunciado.
- Es necesario justificar la solución a cada uno de los ejercicios. Se valorará tanto la corrección de la solución como la justificación dada.

Formato y fecha de entrega

Hay que entregar un único archivo ZIP con las respuestas a todos los ejercicios. El nombre del archivo tiene que ser: *ApellidosNombre_ISCSDPRAC2.zip*.

Este documento se entregará en el espacio de Registro de Evaluación Continua (REC) del aula antes de las **23:59 horas del día 16 de mayo de 2022**. No se aceptarán entregas fuera de plazo.